

Lecture 2: Trees, BST, Heaps & Hash Tables

CS301 — Algorithms & Data Structures

Department of Computer Science

1. Trees

A tree is a hierarchical data structure with a root node and subtrees of children. Trees model hierarchical relationships (file systems, organization charts, DOM).

Terminology: - Root: topmost node with no parent - Leaf: node with no children - Height: length of longest path from root to leaf - Depth: distance from root to a node - Degree: number of children of a node

Binary Tree: each node has at most 2 children (left and right).

Tree Traversals (for binary trees): 1. Inorder (Left, Root, Right): gives sorted output for BST 2. Preorder (Root, Left, Right): used for copying tree 3. Postorder (Left, Right, Root): used for deletion, expression evaluation

All three traversals: $O(n)$ time, $O(h)$ space (h = height, for recursion stack).

2. Binary Search Trees (BST)

A BST is a binary tree where every node satisfies: left child < node < right child

This property enables efficient search.

BST Operations: - Search: $O(h)$ — h is height (best $O(\log n)$ balanced, worst $O(n)$ skewed) - Insert: $O(h)$ - Delete: $O(h)$ — three cases: no child, one child, two children (replace with inorder successor)

BST worst case: inserting sorted data gives a linked list (skewed tree), degrading all operations to $O(n)$.
Solution: Self-balancing BSTs (AVL, Red-Black trees) maintain $O(\log n)$ height.

AVL Tree: height-balanced BST. Balance factor = height(left) - height(right) must be in $\{-1, 0, 1\}$.
Rebalancing uses rotations: single rotation (LL, RR) or double rotation (LR, RL). All operations: $O(\log n)$ guaranteed.

3. Heaps

A heap is a complete binary tree satisfying the heap property. - Max-Heap: parent $\geq$ children (root is maximum) - Min-Heap: parent $\leq$ children (root is minimum)

Heap Operations: - insert: $O(\log n)$ — add to end, bubble up (heapify-up) - extract_max: $O(\log n)$ — remove root, put last element at root, bubble down - peek (max): $O(1)$ - build_heap: $O(n)$ — NOT $O(n \log n)$ — Floyd's algorithm is $O(n)$

Heap Implementation: stored as array (no pointers needed). For node at index i : left child = $2i+1$, right child = $2i+2$, parent = $(i-1)//2$

Heapsort: 1. Build max-heap from array: $O(n)$ 2. Repeatedly extract max and place at end: $O(n \log n)$
Total: $O(n \log n)$, in-place, NOT stable.

Priority Queue: a heap is the natural implementation. Used in Dijkstra's, Prim's, A*.

4. Balanced BSTs vs Heaps

Comparison: Balanced BST vs Heap

Operation	Balanced BST	Heap
Search(x)	$O(\log n)$	$O(n)$
Insert(x)	$O(\log n)$	$O(\log n)$
Delete(x)	$O(\log n)$	$O(\log n)$
Find Min/Max	$O(\log n)$	$O(1)$
Range queries	$O(\log n + k)$	$O(n)$
Sorted output	$O(n)$ inorder	$O(n \log n)$

Key insight: Use a Heap when you only need to repeatedly find/remove the min or max. Use a BST when you need ordered traversal, range queries, or arbitrary deletion.

5. Hash Tables

A hash table stores key-value pairs. A hash function maps keys to array indices.

Hash Function Requirements: 1. Deterministic: same key $\rightarrow$ same hash 2. Uniform distribution: minimize collisions 3. Fast to compute: $O(1)$

Collision Resolution: 1. Chaining: each array slot holds a linked list of all keys that map there. Average case: $O(1)$ with good hash function; worst case $O(n)$ if all keys collide.

2. Open Addressing: find next empty slot using probing. - Linear probing: check next slot - Quadratic probing: check slot $+ 1^2, 2^2, 3^2 \dots$ - Double hashing: use second hash function for step size

Load Factor $\alpha = n/m$ (n = items, m = array size). Performance degrades as $\alpha \rightarrow 1$. Rehash (resize) when $\alpha > 0.7$ typically.

Average case (good hash): $O(1)$ for insert, search, delete. Worst case: $O(n)$ — rare with good hash function.

6. Advanced Topics: Trie

A Trie (prefix tree) is a tree-shaped data structure for storing strings where each node represents a character.

Operations: - Insert word: $O(L)$ where L = word length - Search word: $O(L)$ - Search prefix: $O(L)$ - Delete: $O(L)$

Space: $O(\text{ALPHABET_SIZE} \times L \times N)$ — can be large; use compressed tries or hash maps at each node.

Applications: 1. Autocomplete / search suggestions 2. Spell checking 3. IP routing tables 4. Dictionary implementation